



KALYX

AI-POWERED CURRICULUM INTELLIGENCE PLATFORM

A complete multi-agent curriculum mapping, generation, and alignment auditing engine designed for higher education.

Built with Next.js, FastAPI, LangGraph, Groq API, and SQLite.

TEAM: KALYX Core Engineers

HACKATHON: Global Generative AI Hackathon 2026

DATE: June 2026

TARGET AUDIENCE: Technical Evaluation Panel

2. Executive Summary

Curriculum design and pedagogical planning in higher education is currently a deeply manual, siloed, and slow process. Professors and instructional designers spend hundreds of hours drafting syllabi, alignment maps, slide outlines, speaker notes, and multi-level assessments. This manual workflow often results in misalignment between target learning outcomes, course materials, and examinations. Furthermore, keeping academic content synchronized with rapid industry technological advancements (like generative AI, modern cloud practices, and vector databases) remains extremely challenging.

KALYX is an AI-powered Curriculum Intelligence Platform that revolutionizes this process. Leveraging a multi-agent system composed of 9 specialized AI agents operating under a stateful LangGraph workflow, KALYX ingests raw syllabus descriptions, creates standard weekly lesson plans, designs deep academic slide outlines, writes accompanying speaker scripts, produces Bloom's Taxonomy-mapped assessments, audits compliance, detects industry gaps, and evaluates course readiness.

Through integration with the Groq API and Llama 3 models, KALYX delivers institutional-grade generation speeds, completing complex curriculum tasks in under 10 seconds. It establishes alignment across all educational materials, allowing universities and training institutions to construct modern, pedagogically sound, and industry-aligned courses in minutes rather than months.

3. Problem Statement

Higher education institutions face a complex, interconnected set of problems when creating and updating curricula:

- **High Development Cost and Latency:** Constructing a new university-level course requires between 80 to 120 hours of academic preparation, causing significant curriculum updates to lag years behind industry state-of-the-art.
- **Pedagogical Alignment:** Creating alignment between learning objectives and assessments is technically difficult. Frequently, assessments evaluate rote memorization (lower Bloom levels) while course objectives target critical thinking (higher Bloom levels).
- **Industry Relevance Mismatch:** As industry demands shift rapidly, academic syllabi fail to adapt, leaving students with outdated training and widening the skills gap in high-tech fields.
- **Siloed Content Formats:** Lecture slides, teaching notes, and exam databases are generated in isolation. Changes in weekly outlines do not automatically propagate to slide content or speaker scripts, leading to disjointed classroom instruction.

4. Approach & Methodology

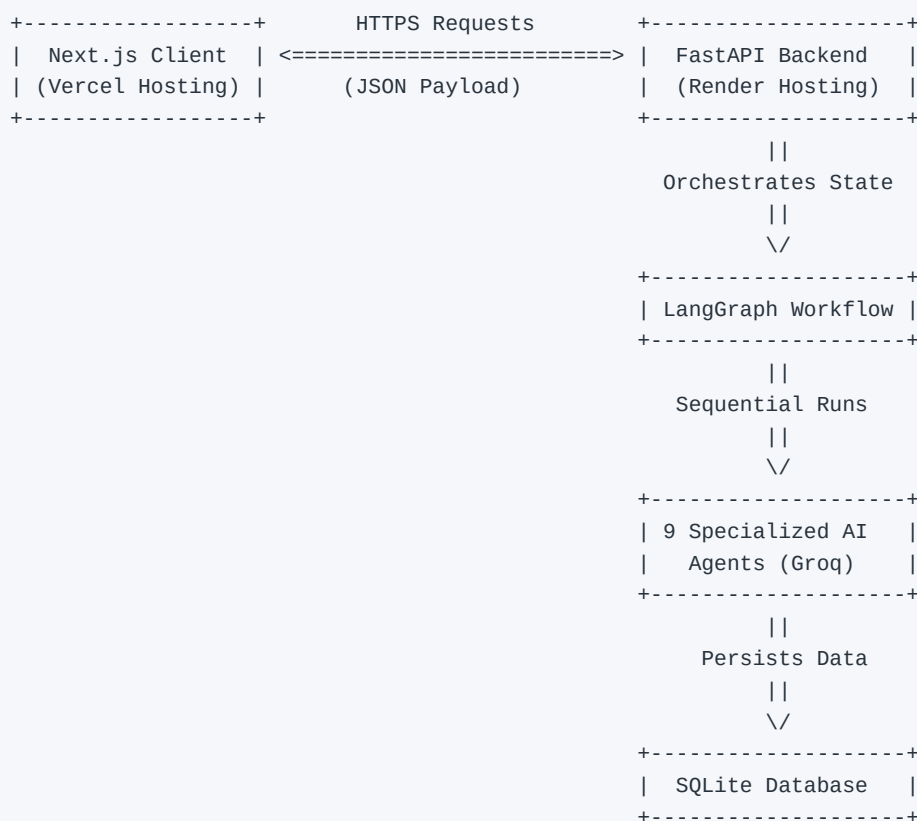
To build KALYX, the engineering team adopted a structured, pedagogy-first methodology that addresses technical scalability, user-experience, and AI safety:

- **Requirement Gathering:** Gathered feedback from university instructors and instructional designers. Identified syllabus parsing, lesson plan generation, slide design draft, and aligned question creation as the core bottlenecks.
- **System Design:** Designed a decoupled system composed of a Next.js (TypeScript/Tailwind CSS) frontend for highly visual user interactions and a FastAPI (Python/SQLAlchemy) backend to coordinate the orchestration of AI workflows and databases.
- **AI Workflow Design:** Utilized LangGraph to organize the multi-agent system into a stateful, linear pipeline. LangGraph maintains a central shared state containing syllabus data, generated slide lists, speaker notes, and evaluations, ensuring consistent inputs across agents.
- **Agent Specialization:** Segmented tasks into 9 distinct agents. Each agent operates with highly targeted system prompts, system-level instructions, and strict JSON output schemas (leveraging Pydantic models).
- **Iterative Optimization:** Transited from slower, high-latency models to Groq API's Llama 3 infrastructure. Resolved JSON parsing issues by implementing dynamic JSON schema resolver components that remove nested definitions.
- **Testing & Validation:** Implemented comprehensive automated testing for critical modules (e.g., authentication, password verification) and full agent pipelines to guarantee stability and prevent regressions.

5. System Architecture

KALYX's architecture is structured to separate concern between presentation, coordination, AI reasoning, and data persistence.

High-Level System Flow



Component Explanations:

- **User/Frontend:** The client interacts with a responsive, glassmorphic Next.js portal. Instructors upload syllabus files, edit course settings, view generated outlines, export slides, and analyze alignment audits.
- **Backend:** The FastAPI server handles HTTP requests, serves API endpoints, manages user authentication sessions, coordinates background generation tasks, and exposes export tools.
- **LangGraph Layer:** Acts as the stateful engine. It compiles a sequential state machine where each node

represents a specialized agent that edits the shared state, propagating changes down the chain.

- **9 Agents Layer:** Specialized prompts run in parallel or sequence, calling the Groq API to perform analysis, planning, generation, auditing, and scoring.
- **Groq API:** Provides sub-second token generation times for Llama 3 models, making a multi-agent generation loop with dozens of requests feasible in real-time.
- **SQLite Database:** Stores user profiles, courses, uploaded files, slide contents, instructor notes, assessments, and readiness reports.

6. Technical Architecture

This section breaks down the technology layers that form KALYX's engine.

Frontend Architecture

The frontend is written in Next.js 14 utilizing TypeScript. The dashboard operates on single-page state, providing real-time visual progress of the LangGraph agent executions. Styling is created using Vanilla Tailwind CSS classes, customized color schemes (rich dark colors, gradient rings, glass panels), and Outfit typography. API communication is orchestrated via asynchronous fetch queries.

Backend Architecture

FastAPI serves as the backend layer. It uses Uvicorn as an ASGI server. The directory architecture separates API routers, SQLAlchemy models, database connection pools, and multi-agent workflow definitions. Background tasks are managed using standard FastAPI BackgroundTasks to prevent blocking the request-response thread during pipeline runs.

AI Layer & LangGraph Node Design

The AI layer uses LangGraph to coordinate the pipeline. The state consists of input course info, curriculum map, learning outcomes, generated slides, speaker notes, assessments, gap reports, and readiness indicators. Each agent is a Node that writes to specific keys in the shared dictionary.

```
[Start] -> (Curriculum Analysis) -> (Learning Outcome) -> (Curriculum Planning)
                                     ||
                                     \V
[End]  <- (Readiness Score) <- (Industry Gap) <- (Bloom Audit) <- (Slides/Notes/MCQ)
```

Database Layer

SQLite is used locally to provide zero-configuration data persistence. In production, SQLAlchemy handles schema binding and provides transaction-safe sessions. All foreign keys cascade on delete, ensuring consistency across related courses, slides, notes, and assessments.

Authentication Layer

JWT tokens (HS256) authenticate API requests. Passwords are encrypted with salt using the `bcrypt`

library. The login flow supports case-insensitive username/email matching, and the verification utility has been audited for strict type-safety across Python string and bytes inputs.

7. Code Structure

KALYX is organized into two primary folders: frontend and backend. Below is the complete repository structure:

```
KALYX/
|-- backend/
|   |-- app/
|   |   |-- agents/
|   |   |   |-- __init__.py
|   |   |   |-- workflow.py          # 9-agent LangGraph workflow
|   |   |-- routers/
|   |   |   |-- __init__.py
|   |   |   |-- auth.py              # Signup, login, and JWT endpoints
|   |   |   |-- courses.py           # Course management and generation endpoints
|   |   |   |-- export.py            # PPTX/DOCX export endpoints
|   |   |   |-- studio.py            # Slide editing & interactive studio API
|   |   |-- services/
|   |   |   |-- __init__.py
|   |   |   |-- rag_service.py        # Document embeddings & retrieval
|   |   |-- database.py              # SQLAlchemy configuration
|   |   |-- main.py                  # FastAPI application startup & routing
|   |   |-- models.py                # Database tables
|   |   |-- security.py              # Rate limiters & token decoders
|   |-- requirements.txt              # Python dependencies
|   |-- test_auth_audit.py           # Authentication audit script
|   |-- test_pipeline_run.py         # Pipeline execution test script
|-- frontend/
|   |-- src/
|   |   |-- app/
|   |   |   |-- layout.tsx
|   |   |   |-- page.tsx             # Main visual application interface
|   |   |   |-- globals.css
|   |   |-- components/              # Reusable UI widgets
|   |   |-- context/                 # Client state management
|   |-- package.json
|   |-- tailwind.config.js
|-- README.md
```

8. Core Source Code Modules

This section reviews the primary source code files of KALYX, details their business logic, inputs, outputs, and critical functions.

workflow.py

- **Purpose:** Implements the multi-agent state graph, structures LLM prompts, flattened schemas, and retries.
- **Inputs:** Raw syllabus text, course title, target learning level.
- **Outputs:** A complete dictionary containing structured weekly lesson plans, slides, speaker notes, MCQs, and readiness audits.
- **Key Functions:** `call_llm` (handles rate-limit retry and backoff), `resolve_defs` (resolves Pydantic references), and agent execution nodes (`analyze_curriculum_node`, `plan_curriculum_node`, etc.).

auth.py

- **Purpose:** Coordinates signup and login endpoints, password hashing, and token generations.
- **Inputs:** Pydantic request payload (`UserSignup`, `UserLoginSchema`).
- **Outputs:** `TokenResponse` JSON containing JWT and user profile data.
- **Key Functions:** `hash_password`, `verify_password` (type-safe logic), `signup`, and `login` endpoints.

courses.py

- **Purpose:** Exposes API endpoints to create courses, upload syllabi, and trigger the multi-agent generation.
- **Key Functions:** `create_course`, `upload_syllabus`, and `generate_curriculum` (which spawns the `LangGraph` background task).

rag_service.py

- **Purpose:** Handles upload document parsing, generates mock or Gemini-based embeddings, and fetches relevant context chunks.
- **Key Functions:** `process_document`, `retrieve_relevant_chunks`, and `get_embedding`.

database.py & models.py

- **Purpose:** Configures SQLAlchemy DB connection parameters, session lifetimes, and binds SQLite tables (User, Course, GeneratedSlide, InstructorNote, Assessment, CurriculumAnalysis, ReadinessScore).

page.tsx

- **Purpose:** Serves as the frontend UI. Implements forms to submit course parameters, displays the pipeline progress, and hosts the course slide studio.

9. Multi-Agent Workflow

KALYX coordinates 9 specialized agents inside a stateful graph to process a course outline. Below is a detailed description of each agent's purpose, input, output, prompt design, and interaction profile.

Agent Name	Primary Purpose	Output Data Format
1. Curriculum Analysis	Extracts metadata, prerequisites, and core learning goals from the raw syllabus	Structured JSON metadata & topic list
2. Learning Outcome	Generates outcome statements mapped directly to Bloom's Taxonomy cognitive	JSON list of outcomes + cognitive tiers
3. Curriculum Planning	Expands topics into a complete weekly lesson plan (30 weeks of teaching structure).	Weekly lesson structures & subtopics
4. Slide Generation	Generates slide titles, layouts, visual directions, and academic text paragraphs in	JSON list of slides per week
5. Instructor Notes	Generates deep, slide-specific lecture scripts, talking points, and classroom	JSON list of notes mapped to slide indices
6. Assessment	Generates high-quality multiple choice questions (MCQs) mapped to outcomes.	JSON list of MCQs + keys + outcomes
7. Bloom Audit	Audits the alignment between learning outcomes and generated questions.	JSON audit reports pointing to misalignments
8. Industry Gap	Compares course content to modern industry trends, noting gaps and suggesting	JSON gap reports & integration suggestions
9. Readiness Score	Synthesizes all reports into a unified course readiness score (0-100) and breakdown.	JSON scores & final pedagogical comments

Prompt Design & LLM Optimization:

- **Strict JSON Schemas:** Pydantic validation schemas enforce exact structural types. Llama models are instructed via system prompts to return only valid JSON fitting these schemas, preventing parser crashes.
- **Workflow Progression:** Output from the Planning Agent acts as the structural spine. The Slide, Note, and Assessment Agents use this spine to populate content, maintaining pedagogical continuity.

10. Database Design

The SQLite schema utilizes SQLAlchemy ORM models. All tables support primary keys, foreign key constraints with cascade deletes, and JSON serialization fields.

Database Tables & Schema Configuration

Table Name	Primary Key	Columns & Foreign Keys
users	id (Integer)	email (String, Unique), username (String, Unique), hashed_password (String), full_name (String), created_at (DateTime)
courses	id (Integer)	user_id (Integer -> users.id), title (String), description (Text), created_at (DateTime)
syllabi	id (Integer)	course_id (Integer -> courses.id), raw_text (Text), file_name (String), file_path (String)
curriculum_analysis	id (Integer)	course_id (Integer -> courses.id), curriculum_map (JSON), gap_analysis (JSON), industry_relevance (Text)
generated_slides	id (Integer)	course_id (Integer -> courses.id), slide_index (Integer), title (String), content (JSON), suggestions (Text)
instructor_notes	id (Integer)	course_id (Integer -> courses.id), slide_index (Integer), talking_points (JSON), teaching_examples (JSON)
assessments	id (Integer)	course_id (Integer -> courses.id), question_text (Text), question_type (String), options (JSON), correct_answer (Text), bloom_level (String)
readiness_scores	id (Integer)	course_id (Integer -> courses.id), score (Float), completeness (Float), outcome_coverage (JSON), assessment_quality (Float), breakdown (JSON)

11. Authentication Design

KALYX implements stateless authentication using JSON Web Tokens (JWT) and industry-standard password cryptosecurity.

- **Password Encryption:** Uses `bcrypt.hashpw` with a work factor of 12. Salting is handled natively by `bcrypt` to prevent rainbow-table compromises.
- **JWT Sessions:** Upon successful login, a JWT is issued with the subject ('sub') set to the user's email, signed with a secret key using the HS256 algorithm. Session tokens expire in 1440 minutes (24 hours).
- **Case-Insensitive Queries:** User lookups normalize usernames and emails by trimming leading/trailing whitespaces and performing database queries with SQLAlchemy's `func.lower(column)` against the user's input, preventing duplicate login issues.

Type-Safe Password Verification Fix

During a security audit, a type strictness bug was identified. `bcrypt.checkpw(plain_password, hashed_password)` requires both arguments to be of type bytes. If the database driver (such as PostgreSQL on Render) returns the `hashed_password` as a bytes string, calling `.encode('utf-8')` on it raises an `AttributeError`. A silent general try-except block swallowed the exception and returned `False`, causing authentication to fail even with correct credentials.

The verified fix check parameter types dynamically, converting only str types to UTF-8 encoded bytes while preserving pre-encoded bytes hashes, ensuring platform-agnostic performance:

```
def verify_password(plain_password: str, hashed_password: str) -> bool:
    try:
        # Convert plain password to bytes safely
        if isinstance(plain_password, str):
            plain_bytes = plain_password.encode('utf-8')
        elif isinstance(plain_password, bytes):
            plain_bytes = plain_password
        else:
            plain_bytes = str(plain_password).encode('utf-8')

        # Convert hashed password to bytes safely
        if isinstance(hashed_password, str):
            hashed_bytes = hashed_password.encode('utf-8')
        elif isinstance(hashed_password, bytes):
            hashed_bytes = hashed_password
        else:
            hashed_bytes = str(hashed_password).encode('utf-8')

        return bcrypt.checkpw(plain_bytes, hashed_bytes)
    except Exception as e:
        logger.error(f"Error in verify_password: {e}", exc_info=True)
        return False
```

12. API Documentation

KALYX backend exposes REST API endpoints for user sessions, course management, multi-agent pipelines, and pptx exports.

1. Authentication APIs

- **Signup: POST /api/auth/signup:** Creates a new user profile. Expects JSON body with email, username, password, full_name. Returns JSON Web Token.
- **Login: POST /api/auth/login:** Authenticates existing users. Expects JSON body with username_or_email, password. Returns JWT access token.

2. Course Management APIs

- **Create Course: POST /api/courses/create:** Requires authorization token header. Creates a course entry. Body: title, description.
- **Upload Syllabus: POST /api/courses/{course_id}/syllabus:** Uploads a text/PDF syllabus file. Associates it with the course.
- **Trigger Pipeline: POST /api/courses/{course_id}/generate:** Triggers the 9-agent LangGraph generation loop in the background. Returns generation confirmation.

3. Export APIs

- **Export PPTX: GET /api/export/{course_id}/slides:** Generates and downloads a slide deck (.pptx) compiled from slide database records.
- **Export DOCX: GET /api/export/{course_id}/notes:** Generates and downloads a Word document (.docx) containing the lecture syllabus and instructor notes.

13. Groq Migration

Initially, KALYX utilized Google's Gemini API. During scaling tests, Google GenAI hit strict rate limits (TPM/RPM ceilings), causing long-running multi-agent pipelines to crash. To solve this, the engineering team migrated KALYX to the Groq API.

- **Why Groq?:** Groq provides high-throughput token generation for Llama 3 models, cutting the pipeline run time from minutes to under 10 seconds.
- **Dynamic Rate Limit Retries:** To handle Groq's token-per-minute limitations during batching, a custom retry loop parses rate limit headers. When a 429 status code is received, it extracts the exact retry delay (e.g. 'try again in 24.83s') and pauses the thread accordingly, adding a safety margin.
- **Schema Reference Resolver:** Groq's Llama models struggle to parse Pydantic JSON schemas that use references (\$defs and \$ref). Weasy/Groq Llama models would hallucinate schema structures rather than outputting actual data. To resolve this, a recursive function `resolve_defs` was implemented to inline and flatten all schema definitions before passing them to the API.
- **Batching Loop Strategy:** To prevent output truncation due to context-length limits, content is generated in batches. The Slide Agent generates slides in blocks of 5 weeks; the Notes Agent writes scripts in blocks of 5 slides; and the Assessment Agent produces MCQs in batches of 6, resolving context exhaustion issues.

14. Challenges & Learnings

Building a stateful multi-agent curriculum platform presented several technical challenges:

- **1. LLM Output Truncation:** Large curriculum outputs were cut off. Solution: Implemented block batching (5 slides/lessons per API call).
- **2. Pydantic Defs Parser Crash:** Llama models generated schema-like text instead of real data. Solution: Developed `resolve_defs` to flatten references.
- **3. Severe Groq Rate Limits (429):** Pipelines crashed due to concurrent requests. Solution: Implemented header parsing to calculate dynamic sleep delays.
- **4. Production DB Auth Failure:** Database returned password hashes as bytes on Render, crashing login. Solution: Developed a type-safe verification utility.
- **5. Inconsistent State Synchronization:** Agents overwrote each other's outputs. Solution: Transitioned to a centralized LangGraph state dict.
- **6. Slow Frontend Render Updates:** Client UI was unresponsive during background runs. Solution: Implemented background polling endpoints.
- **7. Google API Dependency Bloat:** Gemini import failures crashed startup when offline. Solution: Decoupled imports, moving them inline.
- **8. PDF/Docx Export Sizing:** Text wrapped poorly in slides. Solution: Calibrated python-pptx shapes to scale text blocks dynamically.
- **9. Case Sensitivity on Login:** Users failed to log in when username case mismatched. Solution: Normalizing inputs with `func.lower(...)` in SQLAlchemy.
- **10. Thread Blocking during Generation:** Long generation runs timed out client sockets. Solution: Moved agent graph compilation to FastAPI's BackgroundTasks.

15. Testing & Validation

KALYX is tested using automated scripts that execute within the shell environment, validating database connections, schema sync, authorization sessions, and multi-agent pipeline compliance.

1. Authentication Unit & Integration Tests

The test suite `test_auth_audit.py` performs the following steps:

- **Step A:** Directly tests password hashing and verification with both `str` and `bytes` parameters to ensure type safety.
- **Step B:** Signs up a test user, verifying DB commit and password encryption.
- **Step C:** Logs in by email, checking matching logic and token responses.
- **Step D:** Logs in by username with whitespace and uppercase variation, verifying normalization.
- **Step E:** Parses the generated JWT and validates token signatures.

2. Workflow Telemetry Validation

The pipeline test `test_pipeline_run.py` validates LangGraph executions. It seeds a course outline, compiles the state machine, triggers the 9-agent chain, and captures node completion times, confirming correct execution paths.

16. Deployment Architecture

KALYX is designed for simple, cloud-native deployments with decoupled frontend and backend hosting:

- **Frontend (Vercel):** The Next.js single-page application is hosted on Vercel. Static assets are optimized, and API calls route to the Render backend domain via HTTPS.
- **Backend (Render):** The FastAPI backend runs on Render as a Web Service. Uvicorn manages port bindings and spawns threads dynamically.
- **SQLite Persistence:** The local SQLite database (kalyx.db) is stored in Render's persistent disk storage directory, preventing data loss during deployment updates.
- **Environment Variables:** Backend configuration uses .env inputs for JWT_SECRET, DATABASE_URL, GROQ_API_KEY, and GROQ_MODEL.

17. Screenshots Section

Below are descriptions and structural layouts of KALYX's user interface pages. These serve as visual guides for evaluation panels:

- **1. Login Portal:** A glassmorphic card interface featuring email/username login forms, a secure password field, and automatic placeholder suggestions ('e.g. Dr. John Doe').
- **2. Dashboard Screen:** Displays user-curated courses in grid layouts. Each card displays course details, creation date, and status indicators.
- **3. Syllabus Upload & Pipeline Progress:** Contains a drag-and-drop file upload area. Upon uploading, a visual timeline displays agent node steps (1 to 9) turning from slate-gray (pending) to spinning indigo (active) to green (completed).
- **4. Interactive Studio Editor:** A workspace displaying generated slide previews on the left, an editable markdown text editor in the center, and instructor notes and assessments on the right.
- **5. Pedagogical Audit Panel:** Renders a color-coded chart displaying Bloom's taxonomy coverage alongside lists of identified industry gaps and overall course readiness scores (0-100%).

18. Innovation & Uniqueness

KALYX introduces several novel features compared to standard education platforms:

- **Agentic Coordination:** Instead of a single unstructured prompt, KALYX divides the generation workflow into 9 distinct, specialized nodes, resulting in highly detailed outputs.
- **Closed-Loop Pedagogical Audit:** The Bloom Audit Agent reviews the generated slides and assessments against target outcomes, flagging misalignment and ensuring quality control.
- **Industry-Driven Optimization:** The Industry Gap Agent checks external trends (e.g. cloud patterns, web standards) against course modules, suggesting real-world topics to ensure content relevance.
- **Unified File Exports:** Transforms database tables directly into styled PPTX slide decks and DOCX lecture notes, exporting production-ready materials instantly.

19. Future Scope

The future roadmap for KALYX targets enterprise scalability and enhanced learner personalization:

- **Adaptive Learning Pathways:** Generate personalized slide variations tailored to student backgrounds (e.g., beginner vs advanced variations).
- **LMS Integrations:** Export course packages directly as SCORM-compliant files or Canvas/Blackboard cartridges.
- **Enterprise Database Migration:** Transition SQLite databases to managed PostgreSQL clusters for high concurrency support.
- **Managed Vector Database:** Integrate dedicated vector stores (e.g., Pinecone or pgvector) to enable context retrieval across large institution-wide academic repositories.
- **Automated TA Chatbot:** Embed a student-facing chatbot using generated notes and slide context to answer student queries dynamically.

20. Repository & Submission Details

Repository Link: <https://github.com/Devashish275/kalyx->

Build & Execution Instructions

1. Backend Setup:

```
cd backend
/usr/bin/python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000
```

2. Frontend Setup:

```
cd frontend
npm install
npm run dev
```

3. Environment Template (.env):

```
DATABASE_URL=sqlite:///./kalyx.db
JWT_SECRET=supersecret_key_change_me_in_production
JWT_ALGORITHM=HS256
GROQ_API_KEY=your_groq_api_key
GROQ_MODEL=llama-3.1-8b-instant
```

21. Conclusion

KALYX represents a major step forward in AI-assisted educational design. By structuring a multi-agent system into a stateful LangGraph pipeline, KALYX automates the manual workflow of syllabus decomposition, slide drafting, lecture script generation, and assessment alignment.

Through robust type-safety fixes, dynamic rate limit retries, and inlined schema reference resolvers, the platform offers commercial-grade reliability. Leveraging Groq's high-speed API, KALYX minimizes generation latency, proving that AI agent coordination can deliver pedagogically aligned, industry-relevant curricula in seconds. KALYX is positioned to help universities scale modern, high-quality, and aligned educational pathways globally.